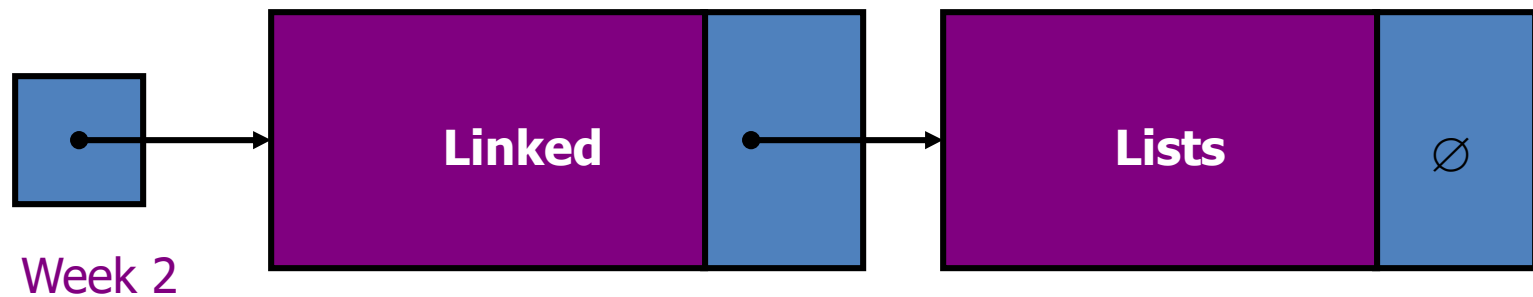


Data Structures & Algorithms (CS-212)



Arrays

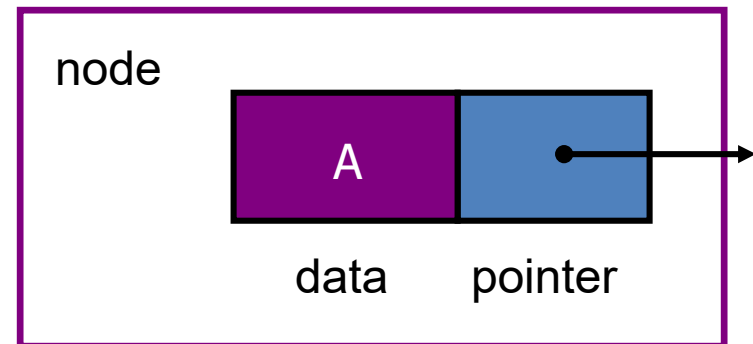
- Pros
 - Simple
 - Useful for storing items in a certain order
- Cons
 - Not very easily adaptable
 - The size n has to be fixed in advance, due to which resizing is also inconvenient
 - Insertion and deletion is problematic because values need to be shifted around

Alternate implementation of sequence

LINKED LIST

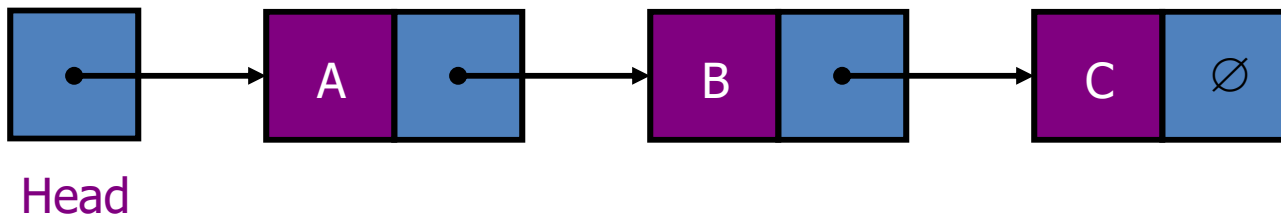
Linked List

- A linked list is a linear data structure where each element is a separate object.
- Collection of nodes that together form a linear ordering
 - Each node stores
 - Data element
 - a pointer to the next node in the list
- Chain the nodes together to form a *linked* list.



Features of Linked List

- *Head*: pointer to the first node
- The last node points to `NULL`



Airport Codes

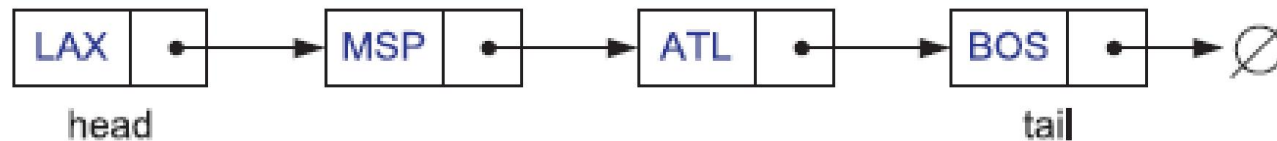


Figure 3.9: Example of a singly linked list of airport codes. The *next* pointers are shown as arrows. The null pointer is denoted by $\emptyset$.

Linked List

- Similar to array in that it holds elements in some sequence
- Unlike an array, it does not have a predetermined size, can be resized by adding or removing nodes

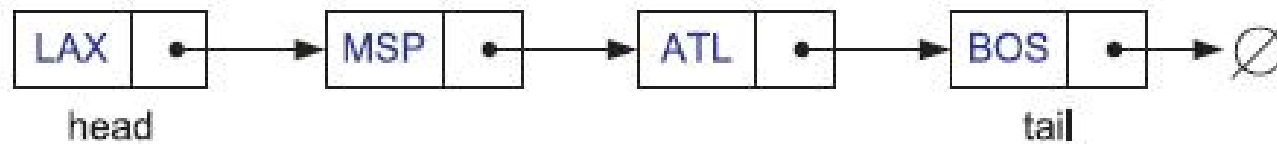


Figure 3.9: Example of a singly linked list of airport codes. The *next* pointers are shown as arrows. The null pointer is denoted by $\emptyset$.

IMPLEMENTATION

A Simple Linked List Class

- We use two classes:
 - **Node**
 - **List**
- **Declare** Node class for the nodes
 - data: **double**-type data in this example
 - next: a pointer to the next node in the list

```
class Node {  
public  
    double data;           // data  
    Node* next;           // pointer to next  
};
```

A Simple Linked List Class

- Declare `List`, which contains
 - `head`: a pointer to the first node in the list.
Since the list is empty initially, `head` is set to `NULL`
 - Operations on `List`

```
class List {  
public:  
    List(void) { head = NULL; }           // constructor  
    ~List(void);                          // destructor  
private:  
    Node* head;  
};
```

A Simple Linked List Class

- Operations of `List`
 - `IsEmpty`: determine whether or not the list is empty
 - `InsertNode`: insert a new node at a particular position
 - `FindNode`: find a node with a given value
 - `DeleteNode`: delete a node with a given value
 - `DisplayList`: print all the nodes in the list

A Simple Linked List Class

- Declare `List`, which contains
 - `head`: a pointer to the first node in the list.
Since the list is empty initially, `head` is set to `NULL`
 - Operations on `List`

```
class List {
public:
    List(void) { head = NULL; }           // constructor
    ~List(void);                          // destructor

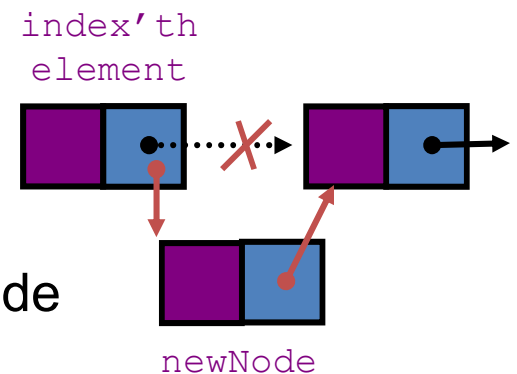
    bool IsEmpty() { return head == NULL; }
    Node* InsertNode(int index, double x);
    int FindNode(double x);
    int DeleteNode(double x);
    void DisplayList(void);
private:
    Node* head;
};
```

Inserting a new node

- `Node* InsertNode(int index, double x)`
 - Insert a node with data equal to `x` after the `index`'th elements.
(i.e., when `index = 0`, insert the node as the first element;
when `index = 1`, insert the node after the first element, and so on)
 - If the insertion is successful, return the inserted node.
Otherwise, return `NULL`.
(If `index` is < 0 or $>$ length of the list, the insertion will fail.)

- Steps

1. Locate `index`'th element
2. Allocate memory for the new node
3. Point the new node to its successor
4. Point the new node's predecessor to the new node



Inserting a new node

- Possible cases of `InsertNode`
 1. Insert into an empty list
 2. Insert in start
 3. Insert at the end
 4. Insert in middle
- But, in fact, only need to handle two cases
 - Insert as the first node (Case 1 and Case 2)
 - Insert in the middle or at the end of the list (Case 3 and Case 4)

Insertion Description

- Insertion at the start of the list
- Insertion at the end of the list
- Insertion in the middle of the list

Insertion Description

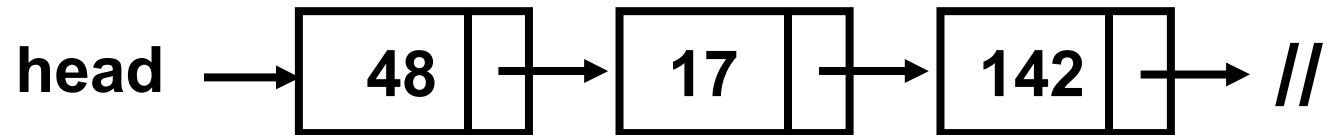
- Insertion at the start of the list
- Insertion at the end of the list
- Insertion in the middle of the list

Insertion at the start

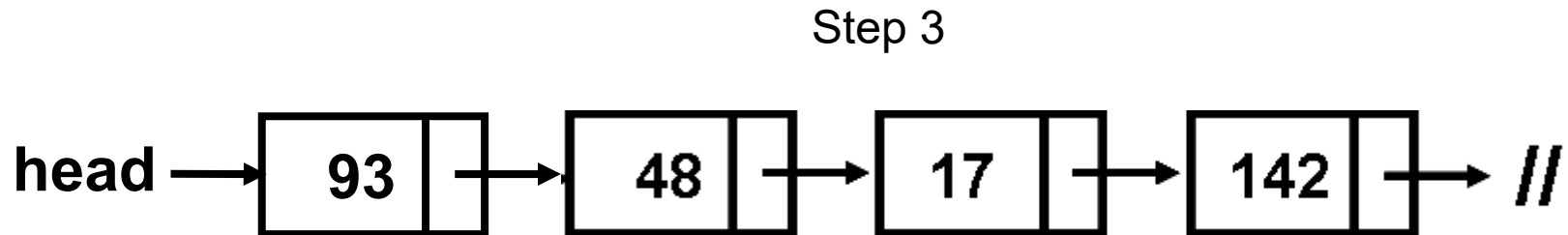
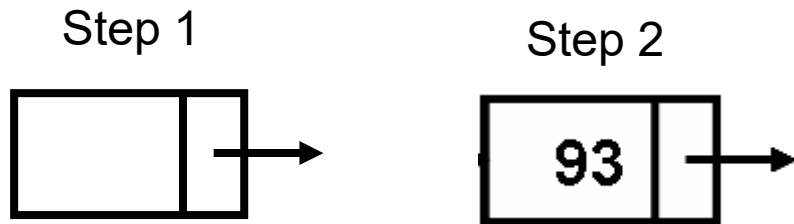
Steps:

- Create a Node
- Set the node data Values
- Connect the pointers

Insertion Description



- Follow the previous steps and we get



Insertion Description

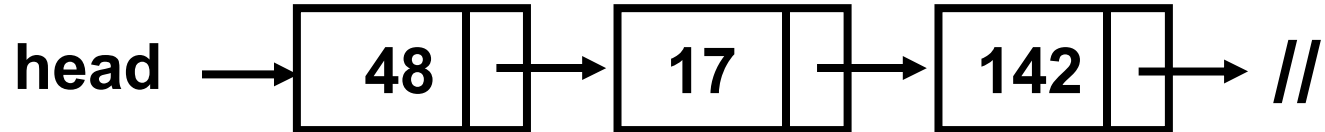
- Insertion at the start of the list
- Insertion at the end of the list
- Insertion in the middle of the list

Insertion at the end

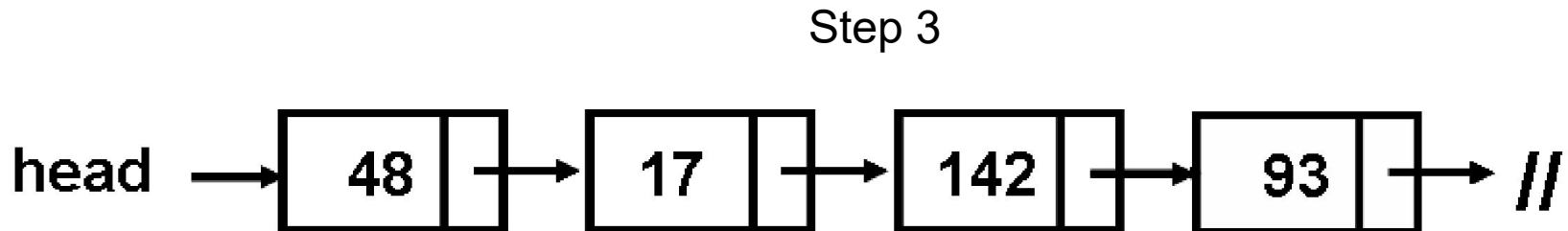
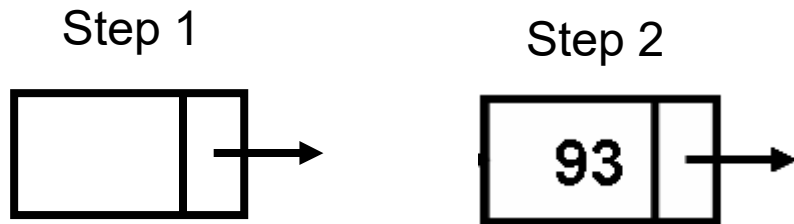
Steps:

- Create a Node
- Set the node data Values
- Connect the pointers

Insertion Description



- Follow the previous steps and we get



Insertion Description

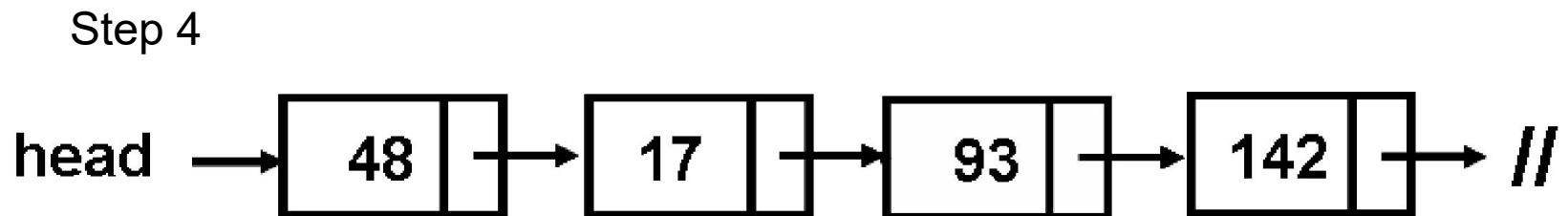
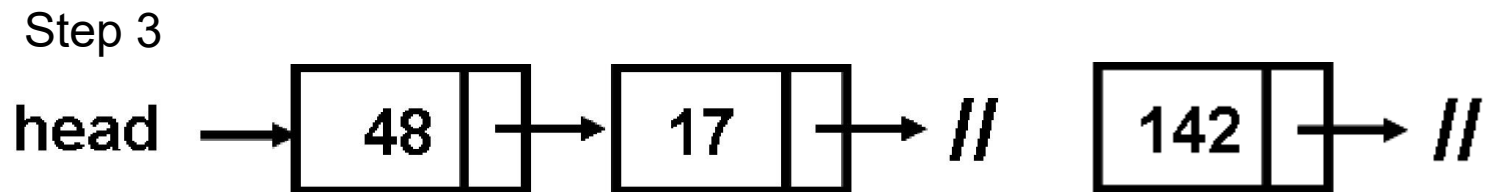
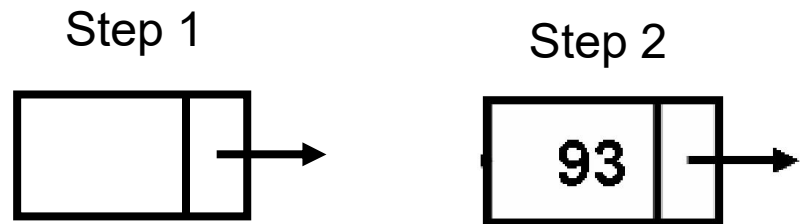
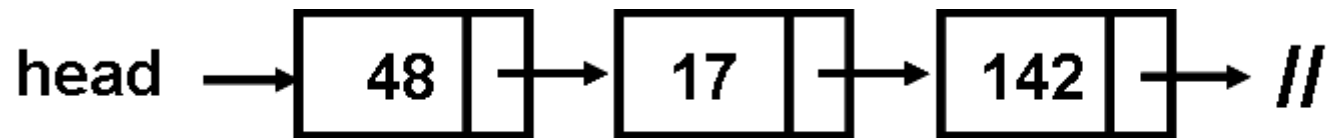
- Insertion at the top of the list
- Insertion at the end of the list
- Insertion in the middle of the list

Insertion in the middle

Steps:

- Create a Node
- Set the node data Values
- Break pointer connection
- Re-connect the pointers

Insertion Description



Inserting a new node

```
Node* List:: InsertNode(int index, double x) {  
    if (index < 0) return NULL;  
  
    int currIndex = 1;  
    Node* currNode = head;  
    while (currNode && index > currIndex) {  
        currNode = currNode->next;  
        currIndex++;  
    }  
    if (index > 0 && currNode == NULL) return NULL;  
}
```

Try to locate index'th node. If it doesn't exist, return NULL.

```
Node* newNode = new Node();  
newNode->data = x;  
if (index == 0) {  
    newNode->next = head;  
    head = newNode;  
}  
else {  
    newNode->next = currNode->next;  
    currNode->next = newNode;  
}  
return newNode;  
}
```

Inserting a new node

```
Node* List::InsertNode(int index, double x) {
    if (index < 0) return NULL;

    int currIndex = 1;
    Node* currNode = head;
    while (currNode && index > currIndex) {
        currNode = currNode->next;
        currIndex++;
    }
    if (index > 0 && currNode == NULL) return NULL;

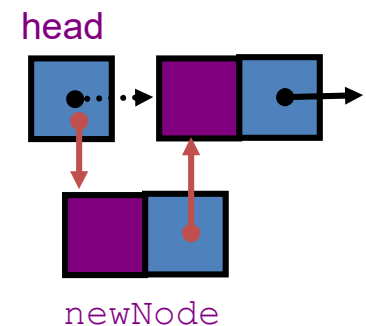
    Node* newNode = new Node();
    newNode->data = x;
    if (index == 0) {
        newNode->next = head;
        head = newNode;
    }
    else {
        newNode->next = currNode->next;
        currNode->next = newNode;
    }
    return newNode;
}
```



Inserting a new node

```
Node* List::InsertNode(int index, double x) {  
    if (index < 0) return NULL;  
  
    int currIndex = 1;  
    Node* currNode = head;  
    while (currNode && index > currIndex) {  
        currNode = currNode->next;  
        currIndex++;  
    }  
    if (index > 0 && currNode == NULL) return NULL;  
  
    Node* newNode = new Node();  
    newNode->data = x;  
    if (index == 0) {  
        newNode->next = head;  
        head = newNode;  
    }  
    else {  
        newNode->next = currNode->next;  
        currNode->next = newNode;  
    }  
    return newNode;  
}
```

Insert as first element



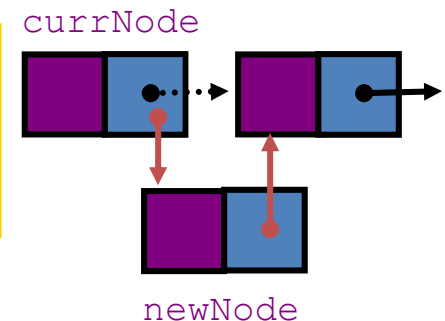
Inserting a new node

```
Node* List::InsertNode(int index, double x) {
    if (index < 0) return NULL;

    int currIndex = 1;
    Node* currNode = head;
    while (currNode && index > currIndex) {
        currNode = currNode->next;
        currIndex++;
    }
    if (index > 0 && currNode == NULL) return NULL;

    Node* newNode = new Node;
    newNode->data = x;
    if (index == 0) {
        newNode->next = head;
        head = newNode;
    }
    else {
        newNode->next = currNode->next;
        currNode->next = newNode;
    }
    return newNode;
}
```

Insert after currNode



Finding a node

- `int FindNode(double x)`
 - Search for a node with the value equal to `x` in the list.
 - If such a node is found, return its position. Otherwise, return 0.

```
int List::FindNode(double x) {  
    Node* currNode      = head;  
    int currIndex = 1;  
    while (currNode && currNode->data != x) {  
        currNode      = currNode->next;  
        currIndex++;  
    }  
    if (currNode) return currIndex;  
    return 0;  
}
```

Deleting a node

- `int DeleteNode(double x)`
 - Delete a node with the value equal to `x` from the list.
 - If such a node is found, return its position. Otherwise, return 0.
- Steps
 - Find the desirable node (similar to `FindNode`)
 - Release the memory occupied by the found node
 - Set the pointer of the predecessor of the found node to the successor of the found node
- Like `InsertNode`, there are two special cases
 - Delete first node
 - Delete the node in middle or at the end of the list

Deletion Description

- Deleting from the start of the list
- Deleting from the end of the list
- Deleting from the middle of the list

Deletion Description

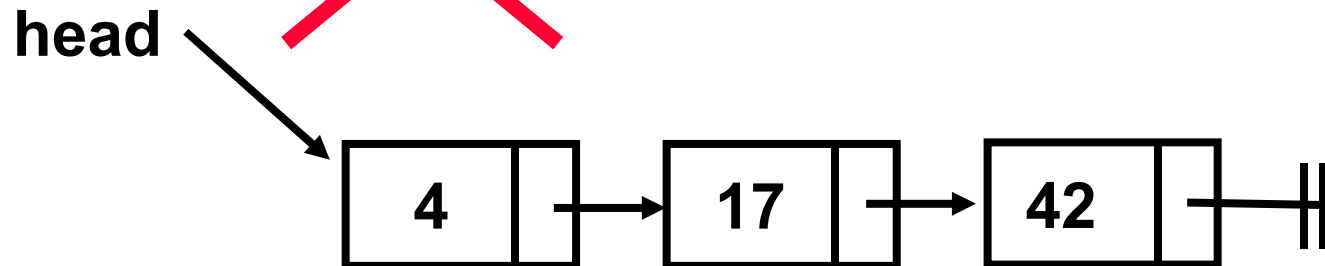
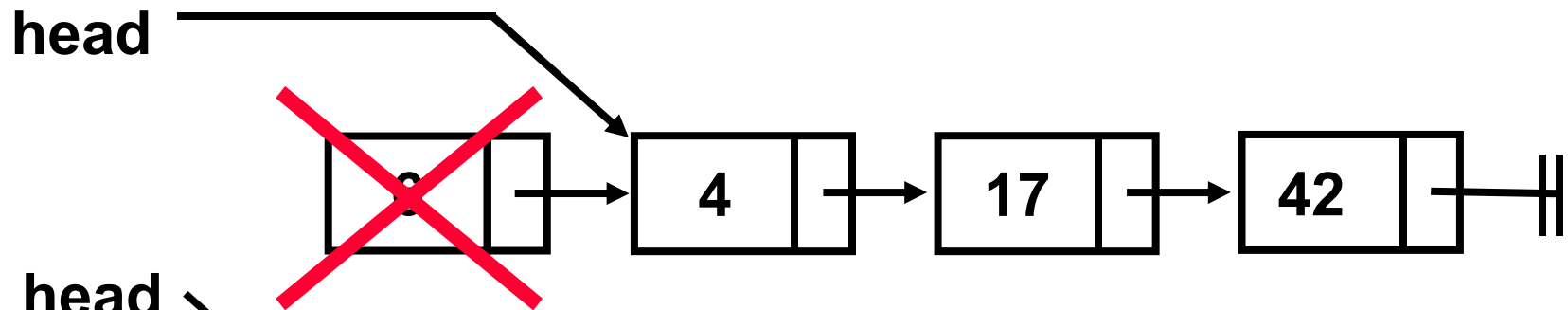
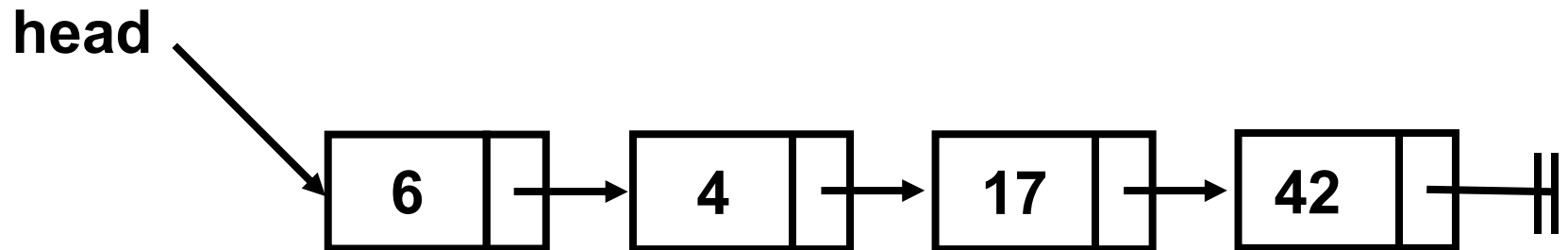
- Deleting from the start of the list
- Deleting from the end of the list
- Deleting from the middle of the list

Deleting from the start

Steps

- Break the pointer connection
- Re-connect the nodes
- Delete the node

Deletion Description



Deletion Description

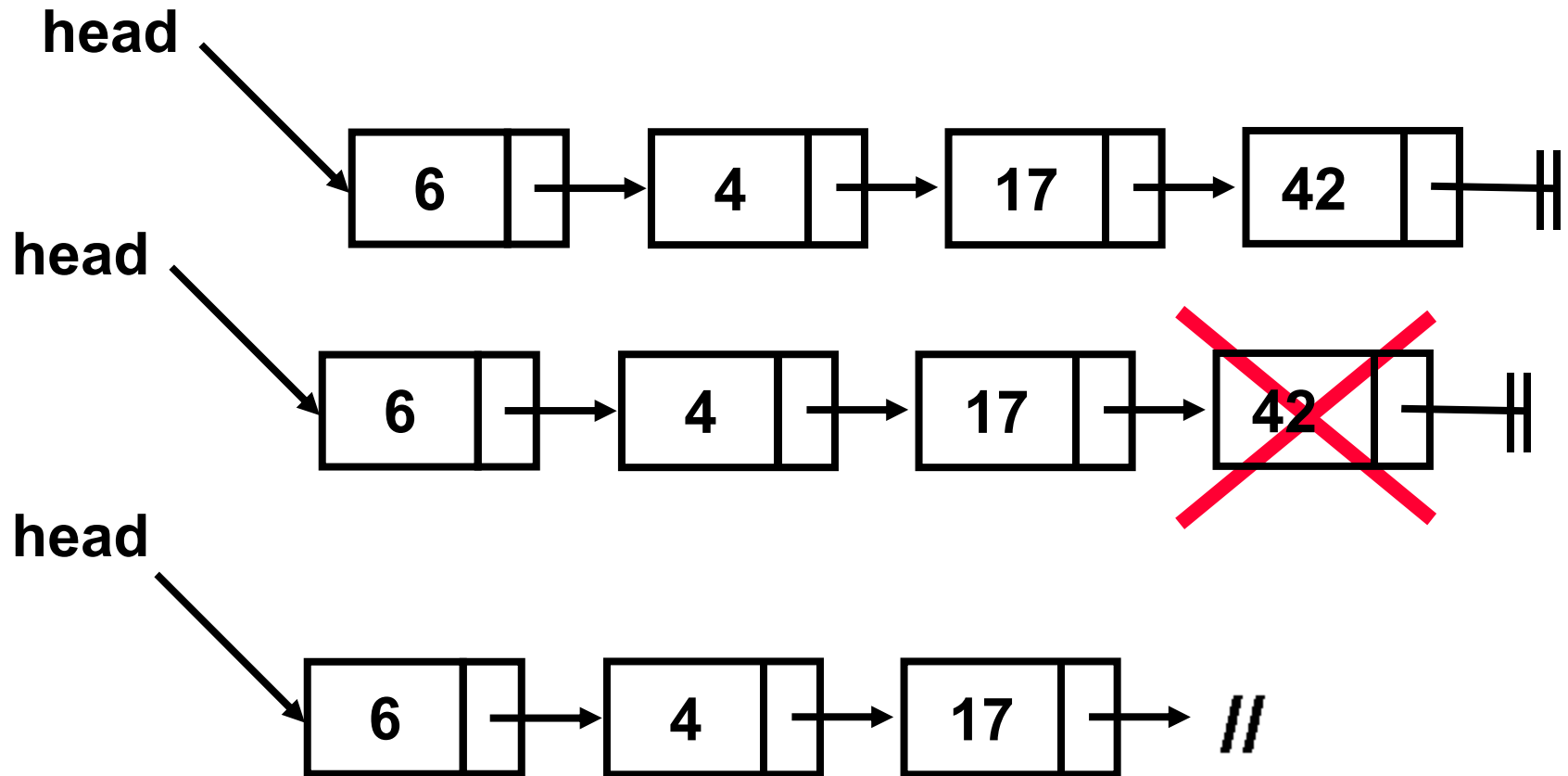
- Deleting from the start of the list
- Deleting from the end of the list
- Deleting from the middle of the list

Deleting from the end

Steps

- Break the pointer connection
- Set previous node pointer to NULL
- Delete the node

Deletion Description



Deletion Description

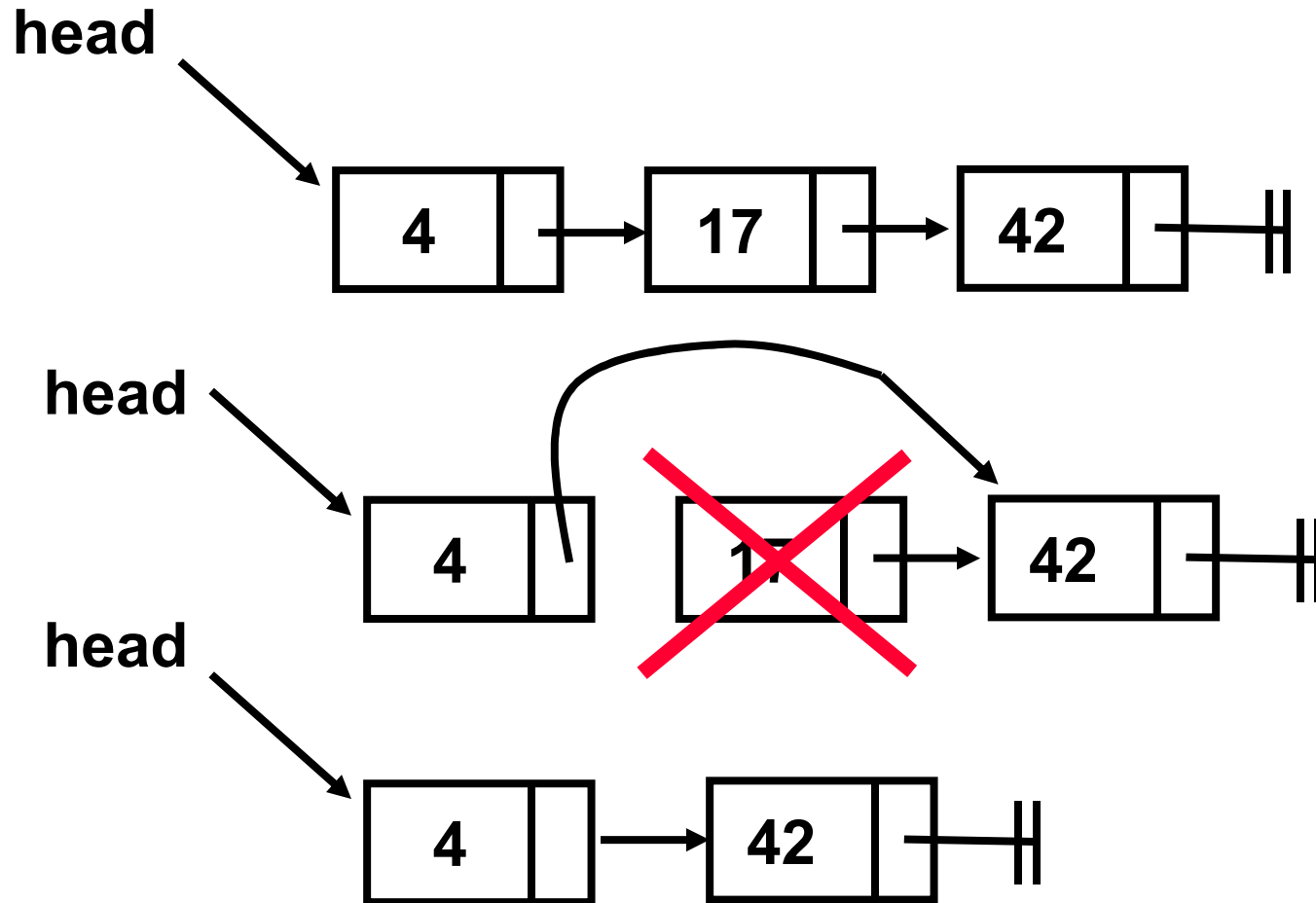
- Deleting from the start of the list
- Deleting from the end of the list
- Deleting from the middle of the list

Deleting from the Middle

Steps

- Set previous Node pointer to next node
- Break Node pointer connection
- Delete the node

Deletion Description



Deleting a node

```
int List::DeleteNode(double x) {
```

```
    Node* prevNode = NULL;
    Node* currNode = head;
    int currIndex = 1;
    while (currNode && currNode->data != x) {
        prevNode = currNode;
        currNode = currNode->next;
        currIndex++;
    }
```

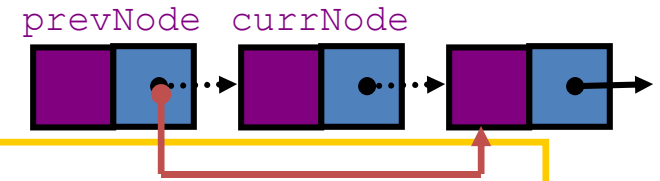
Try to find the node with its value equal to x

```
        if (currNode) {
            if (prevNode) {
                prevNode->next = currNode->next;
                delete currNode;
            }
            else {
                head = currNode->next;
                delete currNode;
            }
            return currIndex;
        }
    }
    return 0;
```

```
}
```

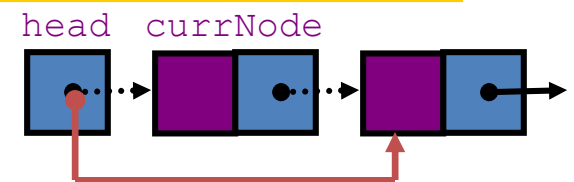
Deleting a node

```
int List::DeleteNode(double x) {  
    Node* prevNode = NULL;  
    Node* currNode = head;  
    int currIndex = 1;  
    while (currNode && currNode->data != x) {  
        prevNode = currNode;  
        currNode = currNode->next;  
        currIndex++;  
    }  
    if (currNode) {  
        if (prevNode) {  
            prevNode->next = currNode->next;  
            delete currNode;  
        }  
        else {  
            head = currNode->next;  
            delete currNode;  
        }  
        return currIndex;  
    }  
    return 0;  
}
```



Deleting a node

```
int List::DeleteNode(double x) {  
    Node* prevNode = NULL;  
    Node* currNode = head;  
    int currIndex = 1;  
    while (currNode && currNode->data != x) {  
        prevNode = currNode;  
        currNode = currNode->next;  
        currIndex++;  
    }  
    if (currNode) {  
        if (prevNode) {  
            prevNode->next = currNode->next;  
            delete currNode;  
        }  
        else {  
            head = currNode->next;  
            delete currNode;  
        }  
        return currIndex;  
    }  
    return 0;  
}
```



Printing all the elements

- `void DisplayList(void)`
 - Print the data of all the elements
 - Print the number of the nodes in the list

```
void List::DisplayList()
{
    int num          = 0;
    Node* currNode   = head;
    while (currNode != NULL) {
        cout << currNode->data << endl;
        currNode      = currNode->next;
        num++;
    }
    cout << "Number of nodes in the list: " << num << endl;
}
```

Destroying the list

- `~List(void)`
 - Use the destructor to release all the memory used by the list.
 - Step through the list and delete each node one by one.

```
List::~~List(void) {  
    Node* currNode = head, *nextNode = NULL;  
    while (currNode != NULL)  
    {  
        nextNode      =      currNode->next;  
        // destroy the current node  
        delete currNode;  
        currNode      =      nextNode;  
    }  
}
```

Using List

```
int main(void)
{
    List list;
    list.InsertNode(0, 7.0);    // successful
    list.InsertNode(1, 5.0);    // successful
    list.InsertNode(-1, 5.0);   // unsuccessful
    list.InsertNode(0, 6.0);    // successful
    list.InsertNode(8, 4.0);    // unsuccessful
    // print all the elements
    list.DisplayList();
    if(list.FindNode(5.0) > 0) cout << "5.0 found" << endl;
    else                       cout << "5.0 not found" << endl;
    if(list.FindNode(4.5) > 0) cout << "4.5 found" << endl;
    else                       cout << "4.5 not found" << endl;
    list.DeleteNode(7.0);
    list.DisplayList();
    return 0;
}
```

6
7
5
Number of nodes in the list: 3
5.0 found
4.5 not found
6
5
Number of nodes in the list: 2

result

Analysis of Linked List

- add
 - we simply insert the new node after the current node. So add is a one-step operation.
- remove
 - remove is also a one-step operation
- find
 - worst-case: may have to search the entire list
- back
 - moving the current pointer back one node requires traversing the list from the start until the node whose next pointer points to current node.

Singly Linked List

- Time consuming to remove any element other than the head, because there is no quick way of accessing the element immediately preceding the one that is to be removed (which needs to be accessed so its next pointer value can be updated)

Lecture content adapted from Michael T. Goodrich textbook,
chapters 3.